Chapter 13 ▪ The Foreign Functions and Memory API

While a “100% Pure Java” solution is nice in principle, there are situations in which you will want to write (or use) code in another language. Such code is usually called *native* code.

Particularly in the early days of Java, many people assumed that it would be a good idea to use C or C++ to speed up critical parts of a Java application. However, in practice, this was rarely useful. A presentation at the 1996 JavaOne conference showed this clearly. The developers of the cryptography library at Sun Microsystems reported that a pure Java platform implementation of their cryptographic functions was more than adequate. It was true that the code was not as fast as a C implementation would have been, but it turned out not to matter. The Java platform implementation was far faster than the network I/O. This turned out to be the real bottleneck.

Since then, the just-in-time compiler has become far better, and it is not common to turn to native code just for performance. But there are other valid reasons, such as these:

- Your application requires access to system features or devices that are not accessible through the Java platform.
- You have substantial amounts of tested and debugged code in another language.

For example, consider using an artificial intelligence library that was written in C or C++, and that uses GPUs. It may not be practical to reimplement it in Java. In such a case, a bridge between Java and native code becomes necessary.

Ever since Java 1.1, the Java platform had an API for interoperating with native C code called the Java Native Interface (JNI).

However, as you will see, there is a certain amount of tedium involved in using JNI. Java 17 introduced, as a preview feature, an API to access “foreign” functions and memory that is quite a bit more convenient than JNI. This FFM API has now become a regular feature of Java 25.

We'll start the chapter by giving you a brief flavor of JNI, and then turn to the FFM API. You should use that API for new projects. At some point in the future, JNI is likely to become deprecated.

Of course, there are drawbacks to going native. If a part of your application is written in another language, you must supply a separate native library for every platform you want to support. Code written in C or C++ offers no protection against overwriting memory through invalid pointer usage. It is easy to write native methods that corrupt your program or infect the operating system.

For that reason, since Java 22, you must opt in to native access (whether through JNI or the FFM API). If you do not use modules, start the program as

```
java --enable-native-access=ALL-UNNAMED ...
```

In a modular program, list the modules for which native access is allowed:

```
java --enable-native-access=module1, module2 ...
```

13.1. Using JNI to Call C Code from a Java Program

Suppose you have a C function that does something you like and, for one reason or another, you don't want to bother reimplementing it in Java. For the sake of illustration, we'll call the C `printf` function with one argument.

With JNI, you need to declare a proxy method using the keyword `native`, and you will obviously need to place the method in a class. The result is shown in **Listing 13.1**.

The `native` keyword alerts the compiler that the method will be defined externally. Of course, native methods will contain no Java code, and the method header is followed immediately by a terminating semicolon. Therefore, native method declarations look similar to abstract method declarations.

Listing 13.1 v2ch13/jnidemo/HelloNative.java

```
1 package v2ch13.jnidemo;
2
3 class HelloNative {
4     public static native int printf(String str);
5 }
```

In this particular example, the native method is also declared as `static`. Native methods can be both static and nonstatic.

You can actually compile this class, but if you try to use it in a program, the virtual machine will tell you it doesn't know how to find `printf`—reporting an `UnsatisfiedLinkError`. To implement the native code, write a corresponding C function. You must name that function *exactly* the way the Java virtual machine expects. Here are the rules:

1. Use the full Java method name, such as `jnidemo.HelloNative.printf`.
2. Replace every period with an underscore, and append the prefix `Java_`. For example, `Java_jnidemo_HelloNative_printf`.
3. If the class name contains characters that are not ASCII letters or digits—that is, `'_'`, `'$'`, or Unicode characters with codes greater than `\u007F`—replace them with `_0xxxx`, where `xxxx` is the sequence of four hexadecimal digits of the character's Unicode value.



Note: If you *overload* native methods—that is, if you provide multiple native methods with the same name—you must append a double underscore followed by the encoded argument types. For example, if you have a native method `greeting()` and another native method `greeting(int repeat)`, then the first one is called `Java_jnidemo_HelloNative_greeting__` and the second, `Java_jnidemo_HelloNative_greeting__I`.

Actually, nobody does this by hand; instead, run `javac` with the `-h` flag, providing the directory in which the header files should be placed:

```
javac -h v2ch13/jnidemo v2ch13/jnidemo/HelloNative.java
```

This command creates a header file `v2ch13_jnidemo_HelloNative.h`, as shown in **Listing 13.2**.

Listing 13.2 v2ch13/jnidemo/v2ch13_jnidemo_HelloNative.h

```
/* DO NOT EDIT THIS FILE - it is machine generated */
#include <jni.h>
/* Header for class v2ch13_jnidemo_HelloNative */

#ifndef _Included_v2ch13_jnidemo_HelloNative
#define _Included_v2ch13_jnidemo_HelloNative
#ifdef __cplusplus
extern "C" {
#endif
/*
 * Class:     v2ch13_jnidemo_HelloNative
 * Method:    printf
 * Signature: (Ljava/lang/String;)I
 */
JNIEXPORT jint JNICALL Java_v2ch13_jnidemo_HelloNative_printf
  (JNIEnv *, jclass, jstring);

#ifdef __cplusplus
}
#endif
#endif
```

As you can see, this file contains the declaration of a function `Java_v2ch13_jnidemo_HelloNative_printf`. (The macros `JNIEXPORT` and `JNICALL` are defined in the header file `jni.h`. They denote compiler-dependent specifiers for exported functions that come from a dynamically loaded library.)

Now, copy the function prototype from the header file into a source file and give the implementation code for the function, as shown in **Listing 13.3**.



Note: You can use C++ to implement native methods. However, you must then declare the functions that implement the native methods as `extern "C"`. (This stops the C++ compiler from "mangling" the method name.) For example,

```
extern "C"
JNIEXPORT int JNICALL Java_v2ch13_jnidemo_HelloNative_printf(JNIEnv* env,
    jclass cl, . . .) { . . . }
```

Listing 13.3 v2ch13/jnidemo/HelloNative.c

```
#include "v2ch13_jnidemo_HelloNative.h"
#include <stdio.h>

JNIEXPORT jint JNICALL Java_v2ch13_jnidemo_HelloNative_printf(
        JNIEnv *env, jclass cl, jstring jstr) {
    const char *cstr = (*env)->GetStringUTFChars(env, jstr, NULL);
    int numchars = printf(cstr);
```

```
    (*env)->ReleaseStringUTFChars(env, jstr, cstr);
    return numchars;
}
```

We will not delve deeply into this code, but there are a couple of highlights.

Each native function receives a parameter `env` that is needed to access the C API for native calls. You can see an example in the call to `(*env)->GetStringUTFChars(...)`.

The `cl` parameter is a descriptor of the Java class. In an instance method, you would instead get a descriptor to the object on which the native method was invoked.

Next, let's look at how to transfer strings to native methods. A native method with a `String` parameter receives a value of an opaque type called `jstring`. Strings are quite different in the two languages: In Java, they are sequences of UTF-16 code points, whereas C strings are null-terminated sequences of bytes. JNI has two sets of functions for manipulating strings: One converts Java strings to "modified UTF-8" byte sequences and another converts them to arrays of UTF-16 values. (The UTF-8, "modified UTF-8," and UTF-16 formats were discussed in **Chapter 2**. Recall that the UTF-8 and "modified UTF-8" encodings leave ASCII characters unchanged, but all other Unicode characters are encoded as multibyte sequences.)



Caution: The standard UTF-8 encoding and the "modified UTF-8" encoding differ for characters with codes higher than `0xFFFF`. In the standard UTF-8 encoding, these characters are encoded as 4-byte sequences. In the "modified" encoding, each such character is first encoded as a pair of "surrogates" in the UTF-16 encoding, and then each surrogate is encoded with UTF-8, yielding a total of 6 bytes. This is clumsy, but it is a historical accident—the JVM specification was written when Unicode was still limited to 16 bits.

In our example, we call `printf` and therefore convert to UTF-8. Had we used the "wide" version `wprintf`, UTF-16 would have been appropriate.

To read the contents of an existing `jstring` object, use the `GetStringUTFChars` function. This function returns a pointer to the "modified UTF-8" characters that describe the character string. Note that a specific virtual machine is free to choose this character encoding for its internal string representation, so you might get a character pointer into the actual Java string. Since Java strings are meant to be immutable, it is *very* important that you treat the `const` seriously and do not try to write into this character array. On the other hand, if the virtual machine uses UTF-16 or UTF-32 characters for its internal string representation, this function call allocates a new memory block that will be filled with the "modified UTF-8" equivalents.

The virtual machine must know when you are finished using the string so that it can garbage-collect it. (The garbage collector runs in a separate thread, and it can interrupt the execution of native methods.) For that reason, you must call the `ReleaseStringUTFChars` function.

Compile the native C code into a dynamically loaded library. The details depend on your compiler.

For example, with the GNU C compiler on Linux, use these commands:

```
gcc -fPIC -I $JAVA_HOME/include -I $JAVA_HOME/include/linux -shared \
    -o v2ch13/jnidemo/libHelloNative.so v2ch13/jnidemo/HelloNative.c
```

With the Microsoft compiler under Windows, the command is

```
cd v2ch13/jnidemo
cl -I %JAVA_HOME%\include -I %JAVA_HOME%\include\win32 ^
    -LD HelloNative.c -FeHelloNative.dll
```

Finally, add a call to the `System.loadLibrary` method in your program. To ensure that the virtual machine will load the library before the first use of the class, use a static initialization block, as in **Listing 13.4**.

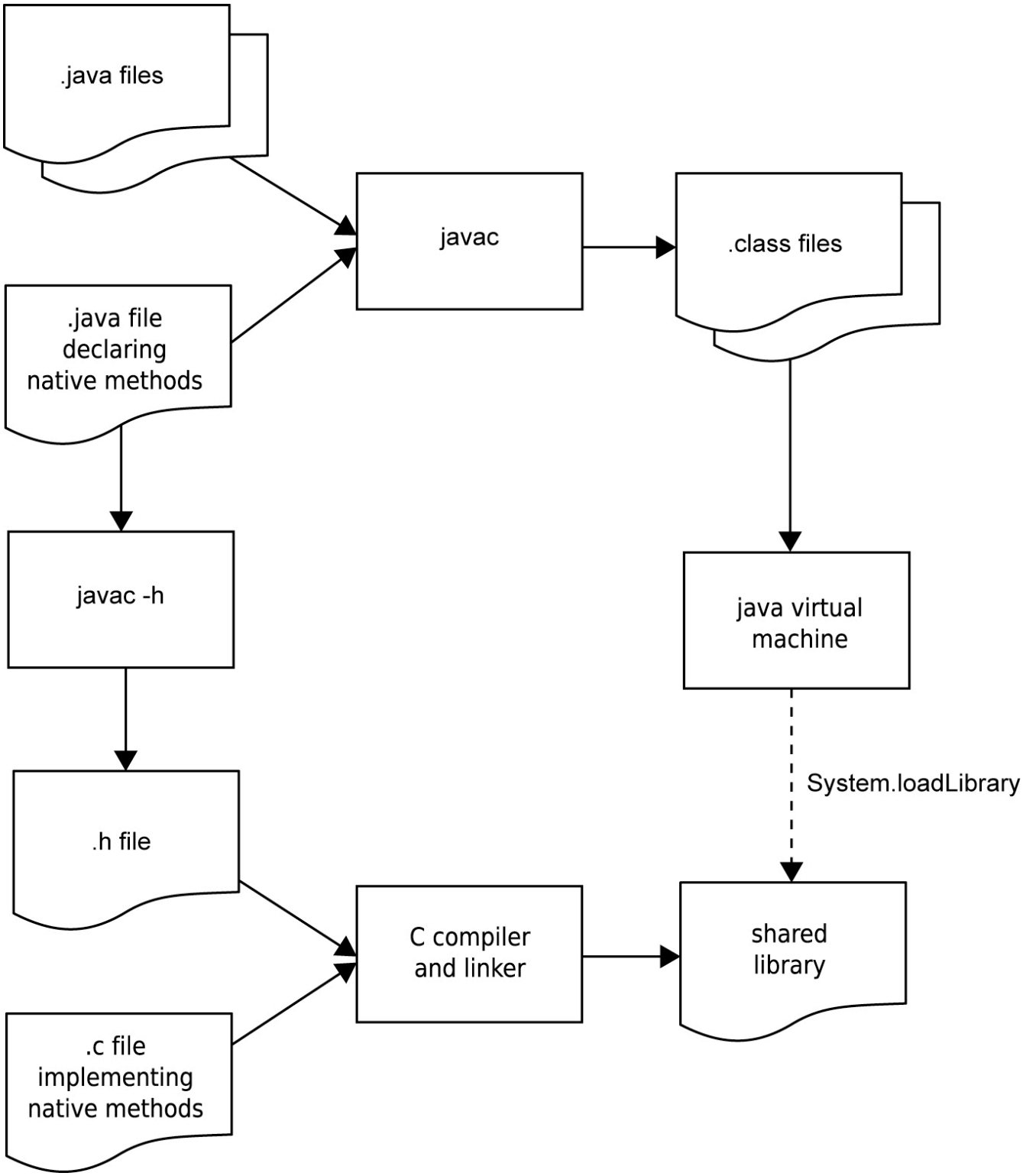


Figure 13.1: Processing native code

Listing 13.4 v2ch13/jnidemo/HelloNativeDemo.java

```
1  package v2ch13.jnidemo;
2
3  class HelloNativeDemo {
```

```
    void main() {
        int result = HelloNative.printf("Hello, Native World!\n");
        IO.println("Printed %d characters.".formatted(result));
    }

    static {
        System.loadLibrary("HelloNative");
    }
}
```

Figure 13.1 gives a summary of the native code processing.

When you run this program, the message "Hello, Native World!" is displayed in a terminal window. This is the result from the C code. The next line, "Printed 21 characters.", is produced by the Java code. We have taken the first step toward bridging the gap between the two languages!



Note: If you run Linux, you must add the current directory to the library path. Either set the `LD_LIBRARY_PATH` environment variable:

```
export LD_LIBRARY_PATH=v2ch13/jnidemo:$LD_LIBRARY_PATH
```

or set the `java.library.path` system property:

```
java -Djava.library.path=v2ch13/jnidemo v2ch13.jnidemo.HelloNativeDemo
```

In summary, with JNI, you follow these steps to link a native method to a Java program:

1. Declare a native method in a Java class.
2. Run `javac -h` to get a header file with a C declaration for the method.
3. Implement the native method in C.
4. Place the code in a shared library.
5. Load that library in your Java program.

`java.lang.System` 1.0

- `void loadLibrary(String libname)`
  loads the library with the given name. The library is located in the library search path. The exact method for locating the library depends on the operating system.

13.2. Using FFM to Call a Foreign Function

When using JNI, you have to write C code to access Java data structures, call the desired C functions, and to convert the result back to Java. Then you have to link the C code into platform-specific libraries. The API for accessing "foreign" functions and memory, developed under the code name "Project Panama," greatly simplifies this process.

The program in Listing 13.5 is a very simple demonstration. It simply calls the `printf` function. No additional C code is necessary. The function is invoked through a `MethodHandle` object.

To access C functions, you first create a `Linker` object. The `Linker` can produce a Java method handle for a C function, given its address and type description.

The API has methods for translating Java objects into memory segments that you can pass to the C function. In the demo, you need to provide a `char*` with the characters from a Java `String`.

The memory is allocated in an `Arena`, so that its deallocation can be managed. Here, we use a *confined* arena. It is autocloseable, and the `close` method deallocates all allocated memory.

To compile and run the demo program, use these commands:

```
javac v2ch13/ffmdemo/HelloForeign.java
java --enable-preview --enable-native-access=ALL-UNNAMED v2ch13.ffmdemo.HelloForeign
```

As you can see, this API is more Java-centric than JNI. All the conversion from Java to C is done in Java code, and you simply invoke the C library function from Java. In the following sections, you will see all the details about linkers, method handles, arenas, and memory segments.

Listing 13.5 v2ch13/ffmdemo/HelloForeign.java

```
1  package v2ch13.ffmdemo;
2
3  import java.lang.invoke.*;
4  import java.lang.foreign.*;
5
6  public class HelloForeign {
7      void main() throws Throwable {
8          Linker linker = Linker.nativeLinker();
9          MethodHandle printf = linker.downcallHandle(
10             linker.defaultLookup().findOrThrow("printf"),
11             FunctionDescriptor.of(ValueLayout.JAVA_INT, ValueLayout.ADDRESS));
12
13         try (Arena arena = Arena.ofConfined()) {
14             MemorySegment str = arena.allocateFrom("Hello, Foreign World!\n");
15             int result = (int) printf.invoke(str);
16             System.out.printf("Printed %d characters.%n", result);
17         }
18     }
19 }
```

13.3. Arenas

When interfacing with foreign functions, the Java part of the program needs to work with *off-heap memory*; that is, memory that was not allocated with a call to `new`.

An *arena* manages the lifetime of off-heap memory segments. You have seen one example in the preceding demo program.

```
try (Arena arena = Arena.ofConfined()) {
    MemorySegment str = arena.allocateFrom(. . .);
    . . .
```

```
} // Arena closed
```

The `Arena` class implements `AutoCloseable`, and the `arena` instance is closed when the `try` block ends. Any allocated segments become invalid afterwards. Attempting to access them causes an exception.

Here, a *confined* arena is used. The memory segments that a confined arena allocates can only be used in the arena's *owner thread*, which is usually the thread in which the arena was created. The arena must also be closed in that thread. Otherwise an exception is thrown.

If memory segments must be shared by multiple threads, you cannot use a confined arena. One option is a *shared* arena, which is alive until you call `close` on it. It is up to you to close the arena at the appropriate time.

Alternatively, you can use an *automatic* arena. The garbage collector will eventually deallocate memory segments when they, and the arena, are no longer referenced. This is convenient, but less predictable. You cannot close an automatic arena.

```
MemorySegment segment = Arena.ofAuto().allocate(. . .);
. . .
segment = null; // Memory eventually deallocated
```

In this example, the automatic arena is only used once, to allocate a single segment.

Finally, the *global arena* is never closed, and the memory segments that it allocates are never deallocated.



Note: You cannot deallocate individual memory segments. To deallocate a memory segment, you need to close the (confined or shared) arena that allocated it.

Table 13.1 summarizes the characteristics of the four arena types.

Table 13.1: Arena Types

Type	Accessible from multiple threads	How closed
Confined		By calling `close`
Shared	✓	By calling `close`
Automatic	✓	By garbage collector
Global	✓	Never

java.lang.foreign.Arena 22

- `static void ofConfined()`
- `static void ofShared()`
- `static void ofAuto()`

  Yields a confined, shared, or automatic arena.

13.4. Memory Segments

A memory segment is a contiguous block of memory. It can be on the Java heap, managed by the garbage collector, or *native*, allocated off the Java heap. An off-heap memory segment has a fixed physical address. A heap segment, just like any Java object, has a reference through which it can be located, but no fixed address since the garbage collector may have moved it. For use with foreign functions, we are generally interested in off-heap memory segments.

You use an arena object to allocate an off-heap segment. As you already have seen, the call

```
String javaString = . . .;
MemorySegment cString = arena.allocateFrom(javaString);
```

encodes a Java string into UTF-8 and allocates a memory segment that holds the bytes, including a null terminator.

To allocate an array, you need to specify the element layout and the number of elements. You will see how to use the `MemoryLayout` interface in **Section 13.5**. For now, we will make use of the layout constants defined in the `ValueLayout` class. For example, here is how to allocate an array of `float` values:

```
MemorySegment audioIn = arena.allocate(ValueLayout.JAVA_FLOAT, sampleCount);
```

If you already have the values in an array, use the `allocateFrom` method:

```
float[] samples = . . .;
MemorySegment audioOut = arena.allocateFrom(ValueLayout.JAVA_FLOAT, samples);
```

When a memory segment is passed to a C function, the function receives the address of the segment. It can then process the contents of the string or array.

Conversely, suppose a C function has filled a memory segment with data. To turn a null-terminated UTF-8 string to a Java string, you pass an offset of 0 to the `getString` method:

```
String javaString = cString.getString(0);
```

To read or write at an array index, provide the memory layout and index:

```
float sample = audioIn.getAtIndex(ValueLayout.JAVA_FLOAT, i);
audioIn.setAtIndex(ValueLayout.JAVA_FLOAT, i, 0f);
```

You can also get a Java array with all values:

```
float[] samples = audioIn.toArray(ValueLayout.JAVA_FLOAT);
```

The `elements` method yields a stream of memory segments, one for each element. You can map each of them into a value and then process the stream further.

```
DoubleStream samples = audioIn.elements(ValueLayout.JAVA_FLOAT)
    .mapToDouble(m -> m.get(ValueLayout.JAVA_FLOAT, 0));
```

The `get` method is like `getAtIndex`, but it takes a byte offset.

Note that the segment does not retain any information about the layout of the data inside. You need to provide the layout whenever you access or update the data.

java.lang.foreign.SegmentAllocator 22

- `MemorySegment allocate(MemoryLayout layout)`
  allocates off-heap memory with the given layout.
- `MemorySegment allocate(MemoryLayout elementLayout, long count)`
  allocates an off-heap array with `count` elements of the given layout.
- `MemorySegment allocateFrom(ValueLayout.ofXxx elementLayout, xxx elements)`
  Allocates off-heap memory holding the given primitive type elements.
- `MemorySegment allocateFrom(String str)`
  allocates off-heap memory holding the null-terminated string with the UTF-8 encoding of `str`.
- `MemorySegment allocate(long byteSize, long byteAlignment)`
  allocates an off-heap memory segment of the given size and alignment constraint.
- `Stream<MemorySegment> elements(MemoryLayout elementLayout)`
  yields a sequential stream of slices with the given layout, taken from this memory segment.

java.lang.foreign.MemorySegment 22

- `String getString(long offset)`
  converts the null-terminated UTF-8 string at the given offset to a Java string.
- `xxx getAtIndex(ValueLayout.ofXxx layout, long index)`
- `void setAtIndex(ValueLayout.ofXxx layout, long index, xxx value)`
  read or write a primitive type value at the given array index.
- `xxx get(ValueLayout.ofXxx layout, long offset)`
- `void set(ValueLayout.ofXxx layout, long offset, xxx value)`
  read or write a primitive type value at the given offset.
- `xxx[] toArray(ValueLayout.ofXxx elementLayout)`
  copies the contents of this memory segment into a primitive type array.

13.5. Memory Layout

In the preceding section, you have seen how to work with memory segments that hold UTF-8 strings and arrays of primitive type values.

In C, you use `struct` for more complex data. For example, the `libsndfile` library for processing sound files (**https://github.com/libsndfile**) yields a value of the following structure when opening a sound file:

```
struct SF_INFO {
    long frames;
    int samplerate;
    int channels;
    int format;
```

```
    int sections;
    int seekable;
} ;
```

You can define a layout for this structure as follows:

```
import static java.lang.foreign.ValueLayout.*;
. . .
StructLayout SF_INFO = MemoryLayout.structLayout(
    JAVA_LONG.withName("frames"),
    JAVA_INT.withName("samplerate"),
    JAVA_INT.withName("channels"),
    JAVA_INT.withName("format"),
    JAVA_INT.withName("sections"),
    JAVA_INT.withName("seekable")
);
```

When matching up the C and Java types, the byte lengths need to match. But unsigned C types are mapped to their sized Java equivalents. If you process those values in Java, you need t to handle them as unsigned values yourself.



Tip: The expression `Linker.nativeLinker().canonicalLayouts()` yields a map from common type names to memory layouts. For example, on a 64-bit platform, `"size_t"` is mapped to a `MemoryLayout` with byte size 8, so you know to use a `JAVA_LONG` to represent the C type `size_t` in Java.

The following types are mapped: `bool`, `char`, `short`, `int`, `long`, `long long`, `float`, `double`, `size_t`, `wchar_t`, `void*`.

Suppose you have a memory segment `info` with the `SF_INFO` layout that was just declared. Here is how you can retrieve one of the components:

```
int channels = info.get(JAVA_INT,
    SF_INFO.byteOffset(PathElement.groupElement("channels")));
```

You have already seen the `get` method, which receives a memory layout and a byte offset. The `StructLayout` can determine the offset of each element. In this case, the offset is 12, since the `"channels"` element is preceded by elements of size 8 and 4.

In order to deal with nested structures, there are a number of interfaces to describe layouts—see **Figure 13.2**. The `ValueLayout` interface has subtypes `ValueLayout.OfInt`, `ValueLayout.ofLong`, and so on, one for each primitive type. The constants `JAVA_INT`, `JAVA_LONG`, and so on in the `ValueLayout` interface are instances of these types. The `AddressLayout` holds a memory address.

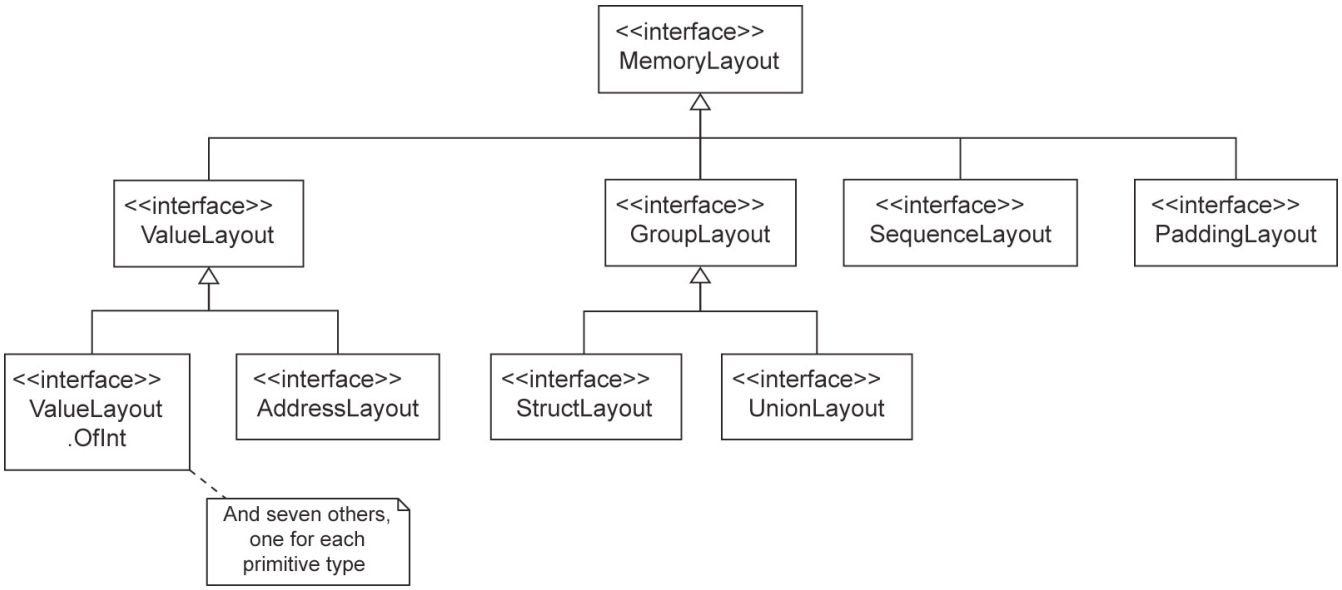


Figure 13.2: The MemoryLayout Hierarchy

As you have seen, a `StructLayout` describes the layout of C structures. Similarly, the `UnionLayout` describes C unions. This is not common, but if you need to deal with unions, you shouldn't have any trouble using the `UnionLayout` interface. It is completely analogous to `StructLayout`.

The `PaddingLayout` describes padding bytes. You sometimes need to insert them to ensure that subsequent elements are properly aligned.

Let us use the `SequenceLayout` to model an array of structures:

```
long count = . . .;
SequenceLayout arrayOfPointLayout = MemoryLayout.sequenceLayout(count,
    MemoryLayout.structLayout(
        JAVA_FLOAT.withName("x"),
        JAVA_FLOAT.withName("y")));
```

To access an individual x- or y-component of the point with index i, we need to compute the byte offset:

```
long offset = arrayOfPointLayout.byteOffset(
    PathElement.sequenceElement(i), PathElement.groupElement("y"));
```

The `byteOffset` method receives a sequence of `MemoryLayout.PathElement` objects that navigate to the selected value.

Now, given a memory segment with that layout, we can get or set the value:

```
MemorySegment arrayOfPoints = Arena.global().allocate(arrayOfPointLayout);
arrayOfPoints.set(JAVA_FLOAT, offset, yvalue);
```

There is more to know about memory layouts and accessing values in memory segments, but what you have seen now suffices to pass complex data between Java and foreign functions. We will take that up in the following section. For more information, see **Section 13.8.1**.

java.lang.foreign.MemoryLayout 22

- `static StructLayout structLayout(MemoryLayout... elements)`
- `static UnionLayout unionLayout(MemoryLayout... elements)`
  create the layout for a `struct` or `union` with the given elements.
- `static SequenceLayout sequenceLayout(long elementCount, MemoryLayout elementLayout)`
  creates the layout for an array of the given length, whose elements have the given layout.
- `static PaddingLayout unionLayout(long byteSize)`
  creates the layout for padding with the given number of bytes.
- `long byteOffset(MemoryLayout.PathElement... elements)`
  yields the offset in this layout for the first value specified by the given path.

java.lang.foreign.MemoryLayout.PathElement 22

- `static MemoryLayout.PathElement groupElement(String name)`
  yields a path to the element with the given name in a group layout.
- `static MemoryLayout.PathElement sequenceElement(long index)`
  yields a path to the element with the given index in a sequence layout.

java.lang.foreign.ValueLayout 22

- `ValueLayout withName(String name)`
  yields a copy of this layout with the given name.

13.6. Looking Up and Invoking Foreign Functions

When compiling C code, a *linker* carries out the essential step of combining the compiled files with precompiled code of library functions, producing an executable file.

The `Linker` abstraction in the FFM API doesn't quite do the same thing. It looks up library functions and prepares `MethodHandle` objects to invoke them.

Actually, the "looking up" part is delegated to a `SymbolLookup` instance. To look up functions in a handful of standard libraries, use a default lookup:

```
Linker linker = Linker.nativeLinker(); // The linker for the underlying platform
SymbolLookup stdlib = linker.defaultLookup();
```

The libraries that the default lookup knows about depend on the platform. On Linux, they include `libc` and `libm`. This lookup suffices for well-known functions such as `printf` and `malloc`.

For use with a specialized library, create a lookup as follows:

```
SymbolLookup sndfile = SymbolLookup.libraryLookup(
    "/lib/x86_64-linux-gnu/libsndfile.so", arena);
```

You specify the path to the library file (as a string or `Path` object), and an arena. The `libraryLookup` method loads the library. When the arena is closed, the library is unloaded.



Note: There is also a *loader lookup*, returned by `SymbolLookup.loaderLookup()`, that looks up functions in libraries loaded through calls to `System.load` and `System.loadLibrary`.

Now you are ready to look up functions:

```
MemorySegment mallocAddr = stdlib.findOrThrow("malloc");
MemorySegment sfOpenAddr = sndfile.findOrThrow("sf_open");
```

The `findOrThrow` method throws a `NoSuchElementException` if the function was not found. There is also a `find` method that returns an `Optional<MemorySegment>` instead. The memory segment contains the address of the function to be invoked.



Note: Many programmers look at the Java API as a model for good design, but in this particular instance, you might want to look twice.

The `find` method returns an `Optional` because surely that is the right thing to do when a lookup might fail. Or is it?

If you are in a similar situation when designing an API, consider whether you want to burden its users with `Optional` results. Of course, a lookup might fail. Java has a perfectly good mechanism for that—an exception.

An `Optional` is appropriate if there is a plausible action to take upon failure, such as substituting a default value, or skipping further processing. But if a library function isn't found, there is unlikely to be a local remedy, and an exception is the best outcome.

The `findOrThrow` method was added later to avoid the tedium of calling

```
MemorySegment mallocAddr = stdlib.find("malloc").orElseThrow();
```

It dispenses with the `Optional`, which is good. But what about the `orThrow` suffix? Fortunately, few exception-throwing methods have such a suffix in their name. The notable exception is `Optional.orElseThrow`, which was given a scary name so that programmers would consider alternatives, such as `orElse` or `map`.

When you find yourself implementing a method whose failure is unlikely to be curable locally, don't return an `Optional` (and of course, don't return `null`). Just throw an exception. Without uglifying the method name.

Having found the memory address of the function to be invoked, we are ready to create a *method handle* for the invocation. A method handle is an object through which you can provide arguments, invoke a method, and obtain the returned value. It is similar to a `Method` object in the `java.lang.reflect` package, but it can also invoke native methods.

To get the method handle, provide the parameter and return types:

```
// void* malloc(size_t)
MethodHandle mallocHandle = linker.downcallHandle(mallocAddr,
    FunctionDescriptor.of(ADDRESS, JAVA_LONG));

// SNDFILE* sf_open(const char *path, int mode, SF_INFO *sfinfo)
MethodHandle sfOpenHandle = linker.downcallHandle(sfOpenAddr,
    FunctionDescriptor.of(ADDRESS, ADDRESS, JAVA_INT, ADDRESS));
```

For a function with return type `void`, you use `FunctionDescriptor.ofVoid` instead:

```
// void free(void *ptr)
MethodHandle freeHandle = linker.downcallHandle(
    stdlib.findOrThrow("free"),
    FunctionDescriptor.ofVoid(ADDRESS));
```

Here, we ask for a *downcall handle* for calling a C function from Java code. In **Section 13.7**, you will see how to use memory handles for calling a Java method from C.



Note: When passing a pointer to a C function, or receiving a pointer result, you use `ValueLayout.ADDRESS`. The type of the memory block is immaterial.



Caution: You must describe the function return type and parameter types accurately. The linker cannot verify this information. If you provide incorrect information, the virtual machine may crash.

Finally, we are ready to invoke the method.

```
MemorySegment block = mallocHandle.invoke(blockSize);

MemorySegment info = arena.allocate(SF_INFO);
MemorySegment cInputPath = arena.allocateFrom(inputFile);
MemorySegment inFile = (MemorySegment) sf_open.invoke(cInputPath, SFM_READ, info);
```

When a foreign function returns a pointer, it is wrapped into a *zero-length* memory segment. To read from the segment, you need some external knowledge of its size and structure.

For example, you know the size of the memory block allocated by `malloc`, so you can set

```
block = block.reinterpret(blockSize);
```



Caution: Clearly, this is a potentially unsafe operation. If you use any memory segment outside its actual bounds, you can corrupt unrelated memory or crash the virtual machine.



Tip: The `jextract` tool, which you can find at **https://github.com/openjdk/jextract**, automatically generates memory layouts and wrappers for foreign calls from C header files.

You now know enough about the FFM API to write nontrivial programs that call a native library. The program in **Listing 13.6** is a typical example. It uses the `libsndfile` library to read a sound file, reverses the samples, and writes the result to another file.

The program first sets up the layout for the `SF_INFO` structure and memory handles for four C functions:

```
SNDFILE* sf_open(const char *path, int mode, SF_INFO *sfinfo)
int sf_close(SNDFILE *sndfile)
sf_count_t sf_read_float(SNDFILE *sndfile, float *ptr, sf_count_t items)
sf_count_t sf_write_float(SNDFILE *sndfile, const float *ptr, sf_count_t items)
```

Then the program opens the input file and retrieves the number of frames and channels from the `SF_INFO` structure.

Next, the call to `sf_read_float` fills a pre-allocated memory segment with the sample data.

For ease of processing in Java, the memory segment is converted to a Java array. After processing, the data is written to the output file.

Listing 13.6 v2ch13/sound/SoundFileReverser.java

```
1  package v2ch13.sound;
2
3  import module java.base;
4
5  import static java.lang.foreign.ValueLayout.*;
6
7  class SoundFileReverser {
8      static final int SFM_READ = 0x10;
9      static final int SFM_WRITE = 0x20;
10
11     void main(String[] args) throws Throwable {
12         String inputFile = args.length > 0 ? args[0] : "sound/input.mp3";
13         String outputFile = args.length > 1 ? args[1] : "sound/output.mp3";
14
15         Linker linker = Linker.nativeLinker();
16         SymbolLookup sndfile = SymbolLookup.libraryLookup(
17             "/lib/x86_64-linux-gnu/libsndfile.so", Arena.ofAuto());
18
19         StructLayout SF_INFO = MemoryLayout.structLayout(
20             JAVA_LONG.withName("frames"),
21             JAVA_INT.withName("samplerate"),
22             JAVA_INT.withName("channels"),
23             JAVA_INT.withName("format"),
24             JAVA_INT.withName("sections"),
25             JAVA_INT.withName("seekable")
26         );
27
28         MethodHandle sf_open = linker.downcallHandle(
29             sndfile.findOrThrow("sf_open"),
30             FunctionDescriptor.of(ADDRESS, ADDRESS, JAVA_INT, ADDRESS)
31         );
32
33         MethodHandle sf_close = linker.downcallHandle(
34             sndfile.findOrThrow("sf_close"),
35             FunctionDescriptor.of(JAVA_INT, ADDRESS)
36         );
37
38         MethodHandle sf_readf_float = linker.downcallHandle(
```

```
            sndfile.findOrThrow("sf_readf_float"),
            FunctionDescriptor.of(JAVA_LONG, ADDRESS, ADDRESS, JAVA_LONG)
        );

        MethodHandle sf_writef_float = linker.downcallHandle(
            sndfile.findOrThrow("sf_writef_float"),
            FunctionDescriptor.of(JAVA_LONG, ADDRESS, ADDRESS, JAVA_LONG)
        );

        try (Arena arena = Arena.ofConfined()) {
            MemorySegment info = arena.allocate(SF_INFO);
            MemorySegment cInputPath = arena.allocateFrom(inputFile);

            // Open sound file
            MemorySegment inFile = (MemorySegment) sf_open.invoke(cInputPath, SFM_READ, info);
            if (inFile == MemorySegment.NULL)
                throw new RuntimeException("sf_open: Failed for " + inputFile);
            long frames = info.get(JAVA_LONG,
                SF_INFO.byteOffset(PathElement.groupElement("frames")));
            int channels = info.get(JAVA_INT,
                SF_INFO.byteOffset(PathElement.groupElement("channels")));
            long totalSamples = frames * channels;

            // Allocate native memory for audio samples
            MemorySegment audioIn = arena.allocate(JAVA_FLOAT, totalSamples);

            // Read samples
            long actualFrames = (long) sf_readf_float.invoke(
                inFile, audioIn, frames);
            if (frames != actualFrames)
                throw new RuntimeException(
                    "sf_readf_float: Expected %d frames but read %d frames."
                    .formatted(frames, actualFrames));

            // Copy native array to Java and process
            float[] audioSamples = audioIn.toArray(JAVA_FLOAT);

            // Reverse the frames
            for (int f = 0; f < frames / 2; f++) {
                for (int c = 0; c < channels; c++) {
                    int i = f * channels + c;
                    int j = (int)((frames - 1 - f) * channels + c);
                    float temp = audioSamples[i];
                    audioSamples[i] = audioSamples[j];
                    audioSamples[j] = temp;
                }
            }

            // Write to sound file
            MemorySegment cOutputPath = arena.allocateFrom(outputFile);
            MemorySegment outFile = (MemorySegment) sf_open.invoke(
                cOutputPath, SFM_WRITE, info);
            if (outFile == MemorySegment.NULL)
                throw new RuntimeException("sf_open: Failed for " + outputFile);

            // Copy Java array to native and write
            MemorySegment audioOut = arena.allocateFrom(JAVA_FLOAT, audioSamples);

```

```
            actualFrames = (long) sf_writef_float.invoke(outFile, audioOut, frames);
            if (frames != actualFrames)
                throw new RuntimeException(
                    "sf_writef_float: Expected to write %d frames but wrote %d frames."
                    .formatted(frames, actualFrames));

            sf_close.invoke(inFile);
            int error = (int) sf_close.invoke(outFile);
            if (error != 0)
                throw new RuntimeException("sf_close: Failed with error code " + error);
            IO.println("Reversed " + inputFile + " to  " + outputFile);
        }
    }
}
```

java.lang.foreign.Linker 22

- `static Linker nativeLinker()`
  yields a linker for the underlying platform.
- `SymbolLookup defaultLookup()`
  yields a symbol lookup for a default set of libraries.
- `Map<String, MemoryLayout> canonicalLayouts()`
  yields a map that holds layouts for data types used by the underlying platform.
- `MethodHandle downcallHandle(MemorySegment address, FunctionDescriptor function, Linker.Option... options)`
  yields a method handle for invoking the function with the given address and function descriptor.

java.lang.foreign.SymbolLookup 22

- `static SymbolLookup libraryLookup(String name, Arena arena)`
- `static SymbolLookup libraryLookup(Path path, Arena arena)`
  load the library with the given name or path and yields a symbol lookup for the library symbols. The library is unloaded when the given arena is closed.
- `Optional<MemorySegment> find(String name)`
- `MemorySegment findOrThrow(String name)`
  yield the symbol with the given name, returning an empty `Optional` or throwing a `NoSuchElementException` if the symbol is not found.

java.lang.foreign.FunctionDescriptor 22

- `static FunctionDescriptor ofVoid(MemoryLayout... argLayouts)`
- `static FunctionDescriptor of(MemoryLayout resLayout, MemoryLayout... argLayouts)`
  yield a function descriptor with the given argument layouts and return type `void`, or the given return layout.

`java.lang.invoke.MethodHandle` 7

- `Object invoke(Object... args)`
  invokes the method handle with the given arguments.

- `MemorySegment reinterpret(long newSize)`
  yields a copy of this memory segment with the given size.

13.7. Callbacks

Sometimes, you call a foreign function that wants to call another function when an event occurs; for example, when input is detected at a microphone or camera. I would prefer to demonstrate this with a realistic example, but there is a fair amount of boilerplate and configuration required. Instead, here is an example with the `qsort` function from the standard library:

```
void qsort(void *base, size_t nmemb, size_t size,
    int (*compar)(const void *, const void *));
```

The last parameter is a pointer to a function that is being called whenever two elements are compared.

We want the following Java method to be called back:

```
static int comp(MemorySegment a, MemorySegment b) {
    return Float.compare(a.get(JAVA_FLOAT, 0), b.get(JAVA_FLOAT, 0));
}
```



Note: As already mentioned, keep in mind that this is a toy example to show the mechanics of a callback. One would not normally split up this work between Java and C.

The following code snippet gets a method handle to the foreign function in the usual way:

```
Linker linker = Linker.nativeLinker();
MethodHandle qsort = linker.downcallHandle(
    linker.defaultLookup().findOrThrow("qsort"),
    FunctionDescriptor.of(ADDRESS, JAVA_LONG, JAVA_LONG, ADDRESS));
```

When invoking the function, we supply an array, its length, the size of each element in bytes, and the address of the callback.

To produce that address, first get a `MemoryHandle` to the static Java method:

```
MethodHandle compHandle = MethodHandles.lookup()
    .findStatic(UpcallDemo.class, "comp",
        MethodType.methodType(
            int.class, MemorySegment.class, MemorySegment.class));
```

This has nothing to do with foreign functions. One can get a method handle to any Java method, and invoke it later.

The linker can generate stub code for a C function that then invokes the method handle of the callback:

```
MemorySegment compStub = linker.upcallStub(
    compHandle,
    FunctionDescriptor.of(JAVA_INT,
        ADDRESS.withTargetLayout(ValueLayout.JAVA_FLOAT),
        ADDRESS.withTargetLayout(ValueLayout.JAVA_FLOAT)),
    arena);
```

Note the `withTargetLayout` calls. They configure the memory layout so that the linker can generate code for configuring the `MemorySegment` instances that the Java callback receives. If you don't do that, the calls to `a.get(JAVA_FLOAT, 0)` and `b.get(JAVA_FLOAT, 0)` in the Java callback will fail because the memory segments `a` and `b` will have size zero.

Finally, you are ready to call the foreign function with the callback address:

```
float[] samples = . . .;
MemorySegment csamples = arena.allocateFrom(JAVA_FLOAT, samples);
qsort.invoke(csamples, samples.length, JAVA_FLOAT.byteSize(), compStub);
```

You will find the complete program in **Listing 13.7**

Listing 13.7 v2ch13/upcall/UpcallDemo.java

```
1  package v2ch13.upcall;
2
3  import module java.base;
4
5  import static java.lang.foreign.ValueLayout.*;
6
7  class UpcallDemo {
8      static int comp(MemorySegment a, MemorySegment b) {
9          return Float.compare(a.get(JAVA_FLOAT, 0), b.get(JAVA_FLOAT, 0));
10     }
11
12     void main() throws Throwable {
13         Linker linker = Linker.nativeLinker();
14         MethodHandle qsort = linker.downcallHandle(
15             linker.defaultLookup().findOrThrow("qsort"),
16             FunctionDescriptor.ofVoid(ADDRESS, JAVA_LONG, JAVA_LONG, ADDRESS));
17
18         MethodHandle compHandle = MethodHandles.lookup()
19             .findStatic(UpcallDemo.class, "comp",
20                 MethodType.methodType(
21                     int.class, MemorySegment.class, MemorySegment.class));
22
23         MemorySegment compStub = linker.upcallStub(
24             compHandle,
25             FunctionDescriptor.of(JAVA_INT,
26                 ADDRESS.withTargetLayout(ValueLayout.JAVA_FLOAT),
27                 ADDRESS.withTargetLayout(ValueLayout.JAVA_FLOAT)),
28             Arena.ofAuto());
29
30         float[] samples = new float[100];
31         RandomGenerator generator = RandomGenerator.getDefault();
32         for (int i = 0; i < samples.length; i++)
```

```
            samples[i] = generator.nextFloat();
        try (Arena arena = Arena.ofConfined()) {
            MemorySegment csamples = arena.allocateFrom(JAVA_FLOAT, samples);
            qsort.invoke(csamples,
                samples.length, JAVA_FLOAT.byteSize(), compStub);
            samples = csamples.toArray(JAVA_FLOAT);
            IO.println(Arrays.toString(samples));
        }
    }
}
```

java.lang.foreign.Linker 22

- `MemorySegment upcallStub(MethodHandle target, FunctionDescriptor function, Arena arena, Linker.Option... options)`
  generates function code, and yields a function pointer through which native code can call the given method handle, using the argument and return layouts given by the function descriptor. The memory for the function code is reclaimed when the arena closes.

`java.lang.invoke.MethodHandles` 7

- `static MethodHandle.Lookup lookup()`
  yields a lookup object for looking up method handles.

`java.lang.invoke.MethodHandle.Lookup` 7

- `MethodHandle findStatic(Class<?> refc, String name, MethodType type)`
  finds a method handle for a static method of the given class with the given name and type.

`java.lang.invoke.MethodType` 7

- `static MethodType methodType(Class<?> rtype, Class<?> ptype0, Class<?>... ptypes)`
  yields a method type with the given return type and parameter types.

13.8. Advanced Topics

You have now seen enough about the FFM API to write nontrivial programs. The API provides solutions for a number of more complex situations that we will examine in the following sections.

13.8.1. Accessing Memory With Variable Handles

A *variable handle* allows getting or setting a single value, or multiple values that are specified by *coordinates*. These could be array indices or map keys, or any other objects that describe the location of a value.

```
VarHandle vh = . . .;
Object value = vh.get(coord1, coord2, . . .);
vh.set(coord1, coord2, . . ., newValue);
```

In **Section 13.5**, you have seen how to access elements in a `struct` by computing the element offset from a layout. We looked at the example of the layout of the `SF_INFO` structure that has a `channels` element.

```
long channelsOffset = SF_INFO.byteOffset(PathElement.groupElement("channels"));
int channels = info.get(JAVA_INT, channelsOffset);
```

Instead of computing offsets, you can use variable handles. For example, here is a variable handle to the `channels` element:

```
VarHandle channelsHandle = SF_INFO.varHandle(PathElement.groupElement("channels"));
```

This variable handle has a single "coordinate"—the memory segment holding the structure. Here is how you look inside a memory segment `info` to read the field value:

```
int channels = (int) channelsHandle.get(info);
```

Using a variable handle isn't any simpler than computing a byte offset, but it is a bit safer. After all, it is easy enough to accidentally use the wrong offset.

Now consider an array layout:

```
long count = . . .;
StructLayout pointLayout = MemoryLayout.structLayout(
    JAVA_FLOAT.withName("x"),
    JAVA_FLOAT.withName("y"));
SequenceLayout arrayOfPointLayout = MemoryLayout.sequenceLayout(count, pointLayout);
```

Here is a variable handle for the x-coordinate of a point in the array:

```
VarHandle xHandle = arrayOfPointLayout.varHandle(
    PathElement.sequenceElement(),
    PathElement.groupElement("x"));
```

The `PathElement.sequenceElement()` call returns an *open path element* which gives rise to another coordinate in the variable handle. Now you can access the `i` th x-coordinate as:

```
MemorySegment pointSegment = . . .;
long i = . . .;
float xi = (float) xHandle.get(pointSegment, i);
```

Now suppose you have an array of pointers, each to a point structure.

```
MemoryLayout arrayOfPointPointersLayout = MemoryLayout.sequenceLayout(count,
    ValueLayout.ADDRESS.withTargetLayout(pointLayout));
```

Note the `withTargetLayout` method that attaches the layout of the memory segment to which the address points. We did not do that in the past with `ADDRESS` because we could not make use of the type information. But there is a special path element for following a pointer:

```
VarHandle xHandle = arrayOfPointPointersLayout.varHandle(
    PathElement.sequenceElement(),
    PathElement.dereferenceElement(),
    PathElement.groupElement("x"));
```

A `MemoryLayout` can also produce a method handle for computing byte offsets, which is potentially convenient for paths with open path elements.

```
MethodHandle xByteOffsetHandle = arrayOfPointLayout.byteOffsetHandle(
    PathElement.sequenceElement(),
    PathElement.groupElement("x"));
```

Now you can compute an offset like this:

```
long offset = (long) xByteOffsetHandle(0, i);
```

The first argument is the base offset, usually zero.

Of course, this doesn't work when the path doesn't have indirections.

13.8.2. Calling a Foreign Varargs Function

In C, functions with variable arguments use a special calling convention. The classic example is

```
int printf(const char *format, ...);
```

In order to call such a function, the `FunctionDescriptor` must know where the fixed arguments end and the variable arguments start.

You must make a separate method handle for each pattern of arguments. For example, suppose you'd like to call

```
printf("Hello, %s! Next year, you'll be %d\n", name, age);
```

You will be passing the format string, another string, and an integer.

```
MethodHandle printf = linker.downcallHandle(
    linker.defaultLookup().findOrThrow("printf"),
        FunctionDescriptor.of(JAVA_INT, ADDRESS, ADDRESS, JAVA_INT),
        Linker.Option.firstVariadicArg(1));
```

The linker option specfies the index of the first variable parameter.

13.8.3. Capturing `errno`

Some C functions signal an error condition by placing an integer error code in a location that can be retrieved with the `errno` macro.

To capture this value, use another linker option:

```
MethodHandle fopen = linker.downcallHandle(
    linker.defaultLookup().findOrThrow("fopen"),
    FunctionDescriptor.of(ADDRESS, ADDRESS, ADDRESS),
    Linker.Option.captureCallState("errno"));
```

Before calling the foreign function, allocate a memory segment for the capture state:

```
StructLayout captureStateLayout = Linker.Option.captureStateLayout();
MemorySegment captureState = arena.allocate(captureStateLayout);
```

Pass that memory segment as the first argument when invoking the method handle:

```
var result = (MemorySegment) fopen.invoke(captureState,
    arena.allocateFrom("woozle.txt"),
    arena.allocateFrom("rw"));
```

Upon return, you can read `errno` from the capture state:

```
long errnoOffset = capturedStateLayout.byteOffset(PathElement.groupElement("errno"));
int errno = (int) capturedState.get(JAVA_INT, errnoOffset);
```

Or, with a variable handle:

```
VarHandle errnoHandle = captureStateLayout.varHandle(PathElement.groupElement("errno"));
int errno = (int) errnoHandle.get(capturedState, 0);
```



Tip: To get the values that can be captured, look at

```
List<MemoryLayout> layouts = Linker.Option.captureStateLayout().memberLayouts();
```

On Linux, you currently only get `errno`.

---

java.lang.foreign.MemoryLayout 22

- `VarHandle varHandle(MemoryLayout.PathElement... elements)`
  yields a variable handle for looking up the value to which the path leads.
- `MethodHandle byteOffsetHandle(MemoryLayout.PathElement... elements)`
  yields a method handle for computing the byte offset of the value to which the path leads.

java.lang.foreign.MemoryLayout.PathElement 22

- `static MemoryLayout.PathElement sequenceElement()`
  yields an "open" path element for selecting one of the elements in the sequence.
- `static MemoryLayout.PathElement dereferenceElement()`
  yields a path element that selects the target layout of an address layout.

java.lang.foreign.AddressLayout 22

- `AddressLayout withTargetLayout(MemoryLayout layout)`
  yields a copy of this layout with the given target layout; that is, the layout of the memory segment to which this address points.

java.lang.foreign.Linker.Option 22

- `static Linker.Option firstVariadicArg(int index)`
  yields a linker option describing the index of the first variadic argument of the function to be linked in a downcall.
- `static Linker.Option captureCallState(String... capturedState)`
  yields a linker option directing a downcall invocation to capture the state items with the given names.
- `static StructLayout captureStateLayout()`
  yields the layout for the capture state.

You have now reached the end of the second volume of *Core Java*, completing a long journey in which you encountered many advanced APIs. I started out with topics that every Java programmer needs to know: streams, XML, networking, databases, and internationalization. I concluded with very technical chapters on security, annotation processing, advanced graphics, and native methods. I hope that you enjoyed your tour through the vast breadth of the Java APIs, and that you will be able to apply your newly gained knowledge in your projects.